CPCS-204

Data Structure Midterm Exam

Fall-22

1st Semester

(Female Section)

Q1. Multiple Choice

1.1) After executed the following code what will be displayed:

```
public static void main(String[] args) {

        int Arr[] = {0, 1, 2, 3, 4, 5, 6, 7, 8};
        int n = 6;
        n = Arr[Arr[n] / 2];
        System.out.println(Arr[n] / 2);

    }
```

a) 4
b) 2
c) 1
d) 3

1.2) In Linear Search, the best-case complexity occurs when:

a) When the element is found at the last index.
b) When the element is found at the first index.
c) When the element is not present in the array.
d) All of the above.

1.3) Assume that helpPtr = head, what will the following recursive method will print:

```
Void func2(LLnode helpPtr){

    if(helpPtr == null)
         return;
   func2(helPtr.next);

  System.out.println(helpPtr.data);
}
```

a) Print all list content.
b) Print all list content except the first node.
c) Will not print anything, infinite recursive.
d) Print all list content in reverse.

1.4) For the following loops, determine the overall complexity.

```
public static void main(String[] args) {

        for (int i = 0; i < 10; i++) {
            for (int j = 0; j < n; j++) {
                for (int k = 0; k < n + 1; k++) {
                    System.out.println(i + "" + j);
                }

            }

        }
    }
```

a) $O(n^2)$
b) $O(n^3)$
c) $O(n)$
d) $O(n \log_2 n)$

1.5) For <u>Unsorted</u> Circular Doubly-Linked list, when will the best-case complexity becomes O(1).

a) Inserting a node at the front & at the end.
b) Inserting a node at the front.
c) Inserting a node at the front & at the end, in addition deletion from the front.
d) Inserting a node at the front & at the end, in addition deletion from the front & the end.

1.6) If you have n array elements, with length m. After performing Binary Search, what will be the Big-oh O(?).

a) O(logn)
b) O(mlogn)
c) O(logm)
d) O(nlogm)

1.7) After executed the following code what will be displayed:

```
Linked list : 20->4->3->6->9
value=8;
static void addnode(int value) {
head = new newNode(head.data,head);}
```

a) 20->3->3->4->9->6
b) 20->20->4->3->6->9
c) 20->4->3->6->9
d) 20->20->4->3->6->9

1.8) A sorted array, int [] Max = {8, 15, 26, 33, 40, 56, 70, 99}. Using the Binary Search algorithm, the maximum number of comparisons required to find a key value in the array is:

a) 2
b) 4
c) 3
d) 1

1.9) What will be the output after executing the following code:

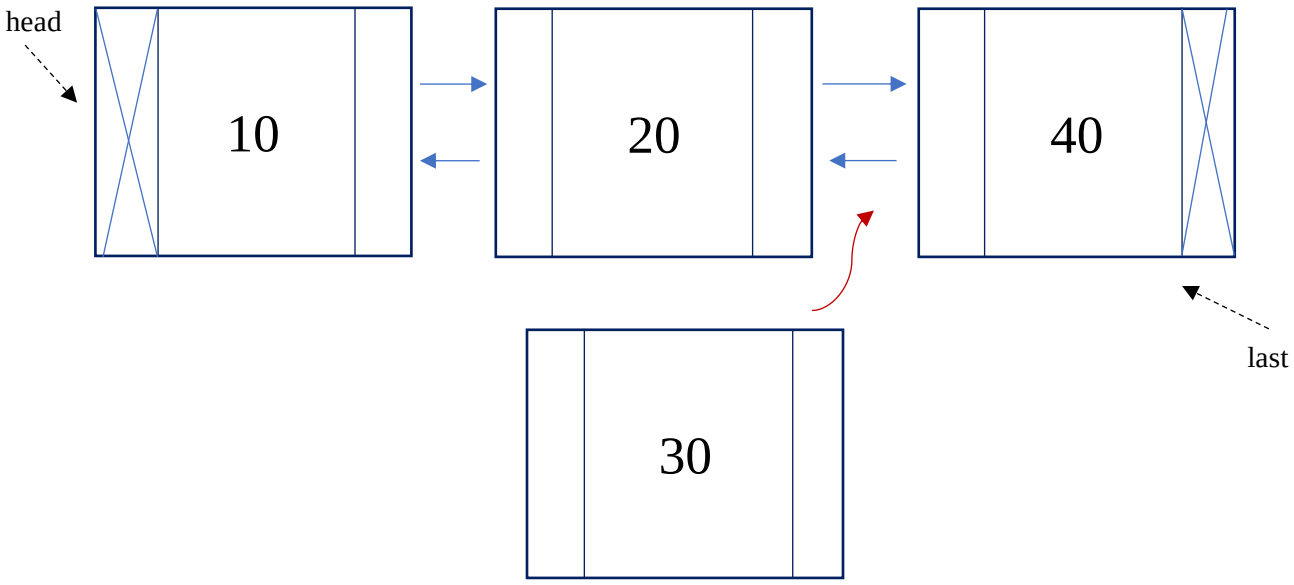
```
public static void main(String[] args) {

    int arr1[]={0,1,2,3,4,5,6,7,8,9};
    int temp;
    for (int i = 0; i <arr1.length; i+=2) {
        temp=arr1[i];
        arr1[i]=arr1[i+1];
        arr1[i+1]=temp;
        System.out.print(arr1[i/2] + 1 + " ");
    } }
```

a) 2 1 4 3 6
b) 2 1 1 3 6
c) 1 2 3 1 6
d) 3 1 2 5 6

Q2. Use the following Double Linked list to answer the following:

2.1) Given a sorted double linked list in increasing order, where head & last, is pointing to first and last node. Insert the node 30 into the list's correct sorted position. No need to traverse.



2.1)

```
DLLnode newNode = ……

newNode.next = ……

newNode.prev = ……

…… = newNode;

…… = newNode;
```

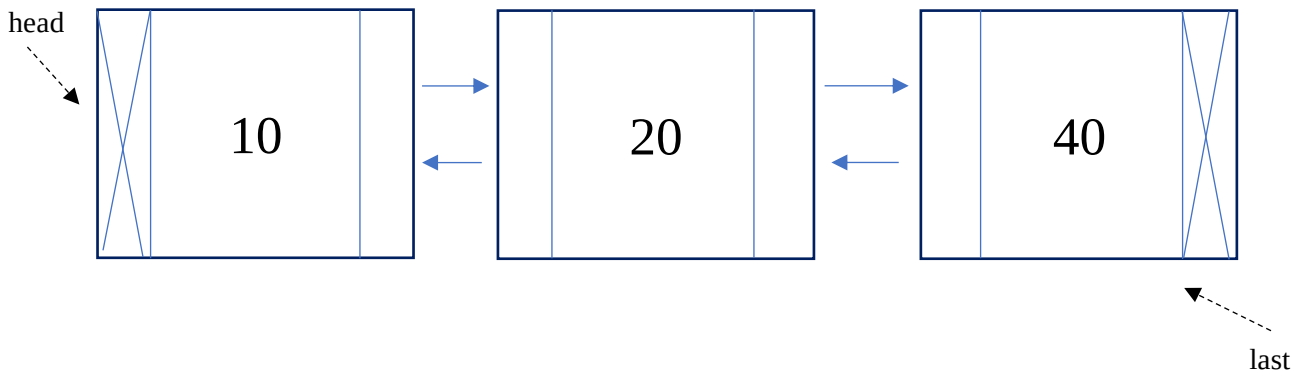
2.2) Using the following Linked list, write segments to delete the last node of the list and update the last pointer.

……
……
……
……
……
……

2.3) Using the following Linked list, write segments to convert Double Linked list to a Circular Doubly Linked list.

……
……
……
……
……
……

Q3. Use the following Double Linked list to answer the following:



3.1) What will the following lines execute:

1. `last.prev.prev.data:` .....

2. `head.next.next.next.data:` .....

3. `last.prev.next.data:` .....

3.2) Suppose a linked list has 6 nodes and head is pointing to the first node in the linked list. What the following statements will do?

```
LLnode helpPtr = head.next;
helpPtr.next = helpPtr.next.next.next;
```

a) It will delete the third & fourth node from the list.
b) It will delete the fourth node from the list.
c) It wil delete the head from the list.
d) Nothing will change in the list.

Q4. Convert the following iterative (loop) to a recursive method:

- **Iterative (loop) method:**

```
Public int Fun(int n){
int d = 0;
for(int i=0; i<n; i++){
  if(n%2 == 0) d = d + (n%10);
n = n/10;
}
 return d;

}
```

- **Recursive method:**

```
if(n%2 == 0)
     return ……
else
     return ……
```

Q5. Given a singly Linked list, complete the method countN() that counts the number of occurrences of the given key in the linked list, return counter if the linked list not empty, otherwise return -1:

Example:

1->3->2->3->8 countN(3) returns 2	4->6->4->2->9->4 countN(4) returns 3	head == null returns -1

```
    static int countN(int value){
      int counter = 0;

      if(……) return ……

      else{
              LLnode helpPtr = head;
              while(……){
                   if(……) counter++:

           ……

    return counter;
}
      }

    }
```

Q6. Given two arrays arr1 [] and arr2 [], compute recursively with NegToZero(int arr1 [], int arr2[], int index) a new array (arr2) with index = 0, where all the negative integers are replaced by zero. For example: arr1 = {1, -2, 2, -3, 4} the new array elements will be arr2 = {1, 0, 2, 0, 4}:

```
static void NegToZero(int arr1 [], int arr2 [], int
index){









}
```

Q7. Trace the following recursive method playWithDigits(int a, int b) and display the output by calling playWithDigits(2,5):

```
 public static void main(String[] args) {

 System.out.println("Result is: " + playWithDigits(2, 5));

    }

 public static int playWithDigits(int a, int b) {
        int temp = a + a;
        if (b == 0) {
            return 0;
        } else if (b % 2 == 0) {
            System.out.println("Play With Digits: (" +
temp + "," + b / 2 + ") ");
            return playWithDigits(a + a, b / 2);
        } else {
            System.out.println("Play With Digits: (" +
temp + "," + b / 2 + ") + " + a);

            return (a) + playWithDigits(a + a, b / 2);
        }

    }
```

OUTPUT:

Q8. Complete the following table (8.1, 8.2):

8.1)

Case	n=2	n=6	n=12	O(?)
A	10ms	10ms	10ms	
B	4ms		4069ms	2^n
C	4ms	36ms	144ms	

8.2) Fastest running time is ……. , comes after it ……., the slowest running time is …….

8.3) The running time for an algorithm is 2ms for input size 4. How long will the algorithm take to run an input size of 8 that runs $O(2^n)$.

……
……
……
……

8.4) Find the upper bound of the given mathematical functions.

	O(?)
f(n) = $5n^3 + n^2 + n$	
f(n) = 3nlogn + n	
6	

Jeem.
GoodLuck <3